

rydcalc 功能简介与 Rydberg 态寿命计算

重点: `total_decay` 如何从态、环境和偶极矩阵元得到寿命

Noise-Tolerant Quantum Control Optimization

2026 年 5 月 13 日

这份 slides 讲什么

- rydcalc 是项目中用于 Rydberg 原子结构、矩阵元、Stark 图、pair interaction 和寿命估算的外部子模块。
- 本项目主要通过 `neutral_yb.external.rydcalc_adapter` 调用它，避免直接改子模块。
- 本报告重点解释：
 - rydcalc 能提供哪些原子物理量；
 - `total_decay(st, env)` 如何自动构造终态并求和；
 - `partial_decay` 中黑体辐射和自发辐射因子如何进入；
 - 对 ^{171}Yb MQDT 态使用时需要注意哪些数值细节。

- 子模块位置: rydcalc/
- 项目适配层: rydcalc_adapter.py
- 当前 notebook 示例:
 - UV Rabi / power estimate notebook;
 - C_3/C_6 estimate notebook;
 - 65S lifetime diagnostic notebook;
 - windowed get_nearby repair notebook;
 - hybrid + residual recalculation notebook。

适配层的主要职责

- 检查 rydcalc 子模块存在;
- 检查 ARC C extension 是否为当前 Python 构建;
- 为 NumPy 2 / SciPy API 差异提供兼容 shim;
- 提供 build_yb171_atom() 入口。

功能	典型 API	本项目用途
原子对象	Ytterbium171, Rubidium	构造物种和能级模型
态构造	atom.get_state(...)	生成 Rydberg / NIST 态对象
矩阵元	get_multipole_me	UV Rabi、跃迁强度、衰变率
Stark 图	analysis_stark	电场混合、能级位移
Pair interaction	analysis_pair_interaction	C_3 , C_6 、blockade 估计
寿命	total_decay, partial_decay	Rydberg decay / blackbody rate

对 ^{171}Yb 而言，核心价值是 MQDT 模型：Rydberg 态不是单一氢样态，而是多个 channel 的线性组合。

典型代码

```
rydcalc = load_rydcalc(require_c_extension=True)
yb = build_yb171_atom(cpp_numerov=True, use_db=False)
st = yb.get_state((65, 0, 1.5, -1.5), tt="vlfm")
env = rydcalc.environment(T_K=300)
tau = 1 / yb.total_decay(st, env)
```

- `tt="vlfm"` 对 Yb171 Rydberg MQDT 态表示 ($\nu/L/F/m_F$) 输入格式。
- `environment` 包含温度、电磁场、LDOS 等外部环境参数。
- `total_decay` 返回总衰变率 Γ ，寿命是 $\tau = 1/\Gamma$ 。

态对象里保存了什么

- 能量: `state.get_energy_Hz()`。
- 角动量量子数: 例如 L, F, m_F 或 n, l, j, m 。
- MQDT channel 列表: `state.channels`。
- channel 权重: `state.Ai` 和相关 MQDT 系数。
- 径向波函数信息: 需要时由 Numerov 或 Whittaker 方法生成。

对寿命计算最关键的接口

$$\text{state.get_multipole_me(other, k=1)} \Rightarrow \langle \text{other} | e r_q | \text{state} \rangle$$

这里返回的是原子单位下的电偶极矩阵元; `partial_decay` 会再转成 SI 单位进入速率公式。

rydcalc.environment 当前保存:

- T_K: 温度, 决定黑体光子占据数;
- Bz_Gauss, Ez_Vcm: 场参数, 主要给 Stark / Zeeman 分析使用;
- ldos(f, q): 局域光学态密度修正, 默认恒等于 1;
- diamagnetism: 是否包含抗磁项。

寿命计算中真正用到的两项

$$T \Rightarrow n_{\text{th}}(\omega, T), \quad \text{LDOS}(f, q) \Rightarrow \text{Purcell/geometry correction.}$$

total_decay 的总体结构

- ① 初始化总速率:

$$\Gamma_{\text{tot}} = 0.$$

- ② 构造一个 `single_basis()`。
- ③ 自动填充可能衰变到的终态。
- ④ 对每个终态 s_f 调用:

$$\Gamma_{i \rightarrow f} = \text{partial_decay}(s_i, s_f, \text{env}).$$

- ⑤ 累加:

$$\Gamma_{\text{tot}} = \sum_f \Gamma_{i \rightarrow f}.$$

返回值

- `printstates=False`: 返回 Γ_{tot} 。
- `printstates=True`: 返回非零 partial decay 列表。
- 寿命不直接返回, 要手动取倒数:

$$\tau = 1/\Gamma_{\text{tot}}.$$

total_decay 的关键代码逻辑

```
gamma_tot = 0
sb = single_basis()

nmax = st.n + 10
sb.fill(st, {
    'dn': st.n,
    'dl': 1,
    'dm': 1,
    'include_fn': lambda s, s0: True if s.n < nmax else False,
})

for s in sb.states:
    pd = self.partial_decay(st, s, env)
    gamma_tot += pd

return gamma_tot
```

这段逻辑说明：total_decay 的物理核心不在传播动力学，而在“列出终态 + 对所有 E1 partial rate 求和”。

自动终态生成的调用链

- `total_decay` 本身不直接知道有哪些终态。
- 它先构造 `single_basis()`，再调用：

```
single_basis.fill(st, include_opts).
```

- `fill` 再调用初态所属 `atom` 的：

```
st.atom.get_nearby(st, include_opts).
```

- 对氢样 / 普通碱金属态，`get_nearby` 的直觉是按 $\Delta n, \Delta l, \Delta m$ 枚举附近主量子数。
- 对 ^{171}Yb MQDT 态，实际逻辑变成按有效量子数 ν 做 MQDT 求根。

Yb171 get_nearby 的关键代码

```
o = {'dn': 2, 'dl': 2, 'dm': 1, 'ds': 0}
for k, v in include_opts.items():
    o[k] = v
if 'df' not in o.keys():
    o['df'] = o['dl']

nu0 = st.nub

for l in np.arange(st.channels[0].l - o['dl'],
                  st.channels[0].l + o['dl'] + 1):
    if l < 0:
        continue
    for f in np.arange(st.f - o['df'], st.f + o['df'] + 1):
        if f < 0:
            continue
        solver = [d for d in self.mqdt_models
                  if d['L'] == l and d['F'] == f][0]['model']

        boundstatesinrange = solver.boundstatesinrange([
            nu0 - o['dn'],
            nu0 + o['dn'],
        ])
```

这段代码说明: `dn` 在 Yb171 路径中直接成为 ν 的搜索半宽; `include_fn` 不参与这里的 `boundstatesinrange` 区间构造。

Yb171 MQDT 终态如何生成

对 ^{171}Yb MQDT 态, `get_nearby` 会:

- 从初态 channel 读出 L_i , 从初态对象读出 F_i, m_F ;
- 按输入的 d_l, d_f, d_m 扫描:

$$L = L_i - \Delta L, \dots, L_i + \Delta L, \quad F = F_i - \Delta F, \dots, F_i + \Delta F.$$

- 对每个 (L, F) 选择对应 MQDT model;
- 调用
 - `solver.boundstatesinrange(...)` 找该 (L, F) manifold 里的束缚态;
 - `solver.channelcontributions(nub)` 计算 MQDT channel mixing;
 - 对允许的 $m_F = m_i - 1, m_i, m_i + 1$ 构造 `state_mqdt`。
- 因此瓶颈不是最后的求和, 而是“先把候选终态全部生成出来”。

一个容易忽略的实现细节

函数签名

```
total_decay(self, st, env, printstates=False, include_fn=lambda s: True)
```

- 签名里有 `include_fn` 参数。
- 但当前实现中，传给 `sb.fill` 的是内部固定 `lambda`：

$$s.n < st.n + 10.$$

- 因此外部传入的 `include_fn` 在当前代码路径下没有真正生效。
- 如果要严格控制终态范围，当前更稳妥的方法是显式构造终态列表，然后逐项调用 `partial_decay`。

问题一：st.n 的语义变了

输入目标态时，我们写的是：

```
initial_state = yb.get_state((65, 0, 1.5, -1.5), tt="vlfm")
```

物理上想表达：

$$|r\rangle = |65\ ^3S_1, F=3/2, m_F=-3/2\rangle.$$

但 Ytterbium171.get_state 会先求 MQDT 根：

```
nutheor = solver.boundstates(nuexp)
nuapprox = round(nutheor * 100) / 100
st = state_mqdt(..., (nuapprox, (-1)**l, f, m), tt="vpfm")
```

对 tt="vpfm", state_mqdt 设置：

$$\text{st.n} = \text{qn}[0] = \nu_{\text{approx}}, \quad \text{st.v} = \nu_{\text{approx}}.$$

所以这里的 st.n 已经不是整数主量子数 65。

目标态对象的实际数值

本地 notebook 中目标态对象打印为:

$|^{171}\text{Yb}:64.56, L=0, F=1.5, -1.5\rangle.$

属性	数值 / 含义
输入标签	$65\ ^3S_1, F = 3/2, m_F = -3/2$
st.qn	(64.56, 1, 1.5, -1.5)
st.n	64.56
st.v	64.56
st.nub	64.56106363
st.v_exact	64.56106363

关键错位

通用 total_decay 把 st.n 当“主量子数 n ”使用; 但 ^{171}Yb MQDT 态里, st.n 是四舍五入后的有效量子数 ν 。

问题二：total_decay 把 st.n 当搜索半宽

total_decay 中的关键代码是：

```
nmax = st.n + 10
sb.fill(st, {
    'dn': st.n,
    'dl': 1,
    'dm': 1,
    'include_fn': lambda s, s0: True if s.n < nmax else False,
})
```

原始设计意图是：

- 若初态是 $n = 65$ ，取 $dn=65$ ；
- 这样就能“从低 n 到高 n ”自动扫一遍可能衰变终态；
- 再用 $s.n < st.n + 10$ 限制最高终态。

对普通主量子数模型，这个写法虽然粗，但语义还算一致。对 Yb171 MQDT， $dn=st.n$ 实际变成：

$$\Delta\nu = 64.56.$$

问题三：get_nearby 直接把 dn 用在 ν 上

^{171}Yb 的 get_nearby 里：

```
nu0 = st.nub
...
boundstatesinrange = solver.boundstatesinrange([
    nu0 - o['dn'],
    nu0 + o['dn'],
])
```

因此对当前态：

$$\nu_0 = 64.56106363, \quad \Delta\nu = \text{st.n} = 64.56.$$

自动搜索区间变成：

$$[\nu_0 - \Delta\nu, \nu_0 + \Delta\nu] = [0.00106363, 129.12106363].$$

这就是搜索范围被扩大的直接代码原因：total_decay 的“从 $n = 0$ 扫到附近态”意图，在 MQDT 表示中变成了“从 $\nu \simeq 0$ 扫到 $\nu \simeq 129$ ”。

问题四：过滤发生得太晚

`single_basis.fill` 的顺序是：

```
st_list = s0.atom.get_nearby(self.s0, include_opts=self.opts)
```

```
for newst in st_list:
    dipole_ok = s0.allowed_multipole(newst, k=1)
    fn_ok = self.opts['include_fn'](newst, s0)
    if (fn_ok) and (dipole_ok or (not self.opts['dipole_allowed'])):
        self.add(newst)
```

- `include_fn` 只在 `get_nearby` 返回之后才执行。
- 因此即使最终过滤条件是

$$s.n < st.n + 10 \approx 74.56,$$

$\nu > 74.56$ 的候选也已经被 MQDT 求根、去重和构造过了。

- `dipole_allowed` 默认是 `False`，所以 `basis` 生成阶段也没有只保留 E1 允许终态。

结论：过滤条件控制的是“最后加入 `basis` 的状态”，不是“MQDT 求根的搜索范围”。

问题五：MQDT 求根本身按细网格扫区间

boundstatesinrange 的核心循环是：

```
for nubguess in np.arange(range[0], range[1], 0.01):
    nutheorb.append(np.round(self.boundstates(nubguess), decimals=accuracy))
...
nutheorb = [
    item for item in nutheorb
    if channelcontributions(item) passes the  $l \leq n_{u_i}$  filter
]
```

- 区间长度越大，boundstates 调用次数线性增加。
- 原生区间长度约为：

$$129.12106363 - 0.00106363 \simeq 129.12,$$

步长 0.01，也就是每个 MQDT model 约 1.3×10^4 个初始点。

- 后续还要对候选根反复调用 channelcontributions。
- 低 ν 区域会触发 sqrt / fractional power 的非有限数值问题。

本地重跑的诊断结果

只测试候选生成，不完整求和，原生 `total_decay` 搜索区间为：

$$[0.00106363, 129.12106363].$$

manifold	候选根数	用时	备注
$L = 0, F = 1/2$	–	> 25 s	单个 solver 超时
$L = 0, F = 3/2$	1041	0.09 s	出现极端非物理 ν
$L = 1, F = 1/2$	395	10.99 s	P 态通道
$L = 1, F = 3/2$	563	18.26 s	P 态通道
$L = 1, F = 5/2$	516	19.96 s	P 态通道

- 这还没有进入所有终态的 `partial_decay` 求和。
- notebook 中直接调用 `yb.total_decay(initial_state, env)` 在 0 K 和 300 K 都 60 s 超时。

问题本质

不是 partial_decay 公式失败

给定一个初态和一个终态，partial_decay 仍然可以正常使用电偶极速率公式计算 partial rate。

真正的问题是自动终态集合

total_decay 的自动 basis 生成逻辑来自更通用的氢样 / 碱金属模型；它把 st.n 当作主量子数和搜索半宽。在 ^{171}Yb MQDT 态中，st.n 实际是有效量子数近似值。

- 语义错位导致搜索区间从 $\nu \simeq 0$ 到 $\nu \simeq 129$ 。
- 过滤条件在 MQDT 求根之后才生效，不能减少前端成本。
- 低 ν MQDT 区域数值病态，容易出现非有限系数或非物理解。

第一版处理方式：显式 P 态 subset

初始 notebook 没有继续依赖原生 `total_decay`，而是显式构造候选终态：

- 第一组诊断：只扫 P 态 MQDT subset,

$$\nu_{\min} = 3.0, \quad \nu_{\max} = \nu_i + 10.$$

- 第二组诊断：把低 n^3P_J 的 NIST 态和高 ν MQDT P 态合并；
- 对每个候选终态单独调用：

$$\Gamma_{i \rightarrow f} = \text{partial_decay}(i, f, \text{env});$$

- 最后按 partial rate 排序，检查主要贡献通道和截断误差。

这种方法换来了可控的终态范围和可诊断的主要 decay channels；但用户指出寿命明显偏长，说明第一版仍漏掉了重要 channel。

在 rydcalc 基础上做的改进

后续 notebook 没有修改 rydcalc 子模块源码，而是在调用层补了三层逻辑：

- ① **分窗** get_nearby: 保留 total_decay 的“动态生成终态”思想，但把 MQDT ν 区间切成小窗口。
- ② **NIST 低态补充**: 把 Yb171_NIST.txt / Yb174_NIST.txt 中的低 lying E1 终态加入候选池。
- ③ **residual sink**: 对文献有总寿命的情形，把有限 channel 仍未解释的 rate 作为有效损失通道单独报告。

原则

$$\Gamma_{\text{model}} = \Gamma_{\text{NIST}} + \Gamma_{\text{MQDT}} + \Gamma_{\text{res}}.$$

其中 Γ_{res} 只用于总损失校准，不当作真实 resolved final state。

改进一：分窗 get_nearby

原生 total_decay 一次请求：

$$\nu \in [0.00106363, 129.12106363].$$

notebook-local 修复是构造一个浅拷贝 proxy_state：

```
proxy.nub = window_center
atom.get_nearby(proxy, {
    "dn": window_half_width,
    "dl": 1,
    "df": 1,
    "dm": 1,
})
```

- 仍然由 get_nearby 动态找 MQDT 终态；
- 不再一次扫到极低 ν 病态区域；
- 相邻窗口有 overlap，再用 $(E, F, m_F, \text{parity})$ 去重；
- 对 65S 目标态，窗口法可稳定生成 1680 个 MQDT P 态候选。

改进二：NIST 低态补充与去重

第一版只靠 MQDT P 态时，低 lying 态贡献不足或能级不够可靠。

- 对 ^{171}Yb 65S:
 - 显式加入 Yb171_NIST.txt 中低 nP 态；
 - 对每个低 P 态枚举允许的 F, m_F ；
 - 使用 `allowed_multipole(..., k=1)` 保留 E1 通道；
 - 高 Rydberg P 态使用 MQDT，实际取 $\nu \geq 12$ 。
- 对 ^{174}Yb Fang benchmark:
 - 使用 Yb174_NIST.txt 中的 singlet 低态；
 - 与 windowed MQDT 高态合并。

去重策略

相同或近似相同终态按 (E, F, m, parity) 合并；低能区优先使用 NIST 能级，高能 Rydberg 区使用 MQDT 态。

改进三：residual sink

有限候选池即使加入 NIST 低态，仍可能少算：

- 其它未解析 manifold；
- perturber / configuration-interaction 相关通道；
- 实验条件下的 trap-induced loss 或 ionization；
- 当前模型没有显式表示的黑体诱导转移。

因此如果有文献总寿命 τ_{ref} ，定义：

$$\Gamma_{\text{ref}} = \frac{1}{\tau_{\text{ref}}}, \quad \Gamma_{\text{res}} = \max(0, \Gamma_{\text{ref}} - \Gamma_{\text{explicit}}).$$

在动力学模型中的含义

$$L_{\text{res}} = \sqrt{\Gamma_{\text{res}}} |\text{sink}\rangle \langle r|.$$

它只保证总人口损失率正确，不提供真实 branching ratio。

对一个初态 i 和终态 f , 先计算角频率:

$$\omega = 2\pi(E_i - E_f).$$

- $\omega > 0$: 向低能态辐射, 包含自发辐射与受激辐射因子 $1 + n_{\text{th}}$ 。
- $\omega < 0$: 向高能态吸收黑体光子, 只有热光子因子 n_{th} 。
- $\omega = 0$: 代码用一个很大的占据数兜底, 正常物理通道中应避免这种情况。

黑体占据数:

$$n_{\text{th}}(\omega, T) = \frac{1}{\exp\left(\frac{\hbar|\omega|}{k_B T}\right) - 1}.$$

partial_decay 的速率公式

电偶极矩阵元由

$$d_{if} = \text{st.get_multipole_me}(\text{other}, k=1)$$

给出，单位是 ea_0 。代码中的速率为：

$$\Gamma_{i \rightarrow f} = \rho_{\text{LDOS}}\left(\frac{|\omega|}{2\pi}, \mathbf{q}\right) \cdot \eta(\omega, T) \cdot |\omega|^3 |d_{if}|^2 \frac{(ea_0)^2}{3\pi\epsilon_0\hbar c^3}.$$

其中

$$q = m_f - m_i, \quad \eta(\omega, T) = \begin{cases} 1 + n_{\text{th}}, & \omega > 0, \\ n_{\text{th}}, & \omega < 0. \end{cases}$$

矩阵元内部如何处理 MQDT 态

get_multipole_me 对 MQDT 态不是简单查表，而是对 channel 逐项求和：

$$|i\rangle = \sum_{\alpha} A_{\alpha}^{(i)} |\alpha\rangle, \quad |f\rangle = \sum_{\beta} A_{\beta}^{(f)} |\beta\rangle.$$

对每对 channel：

- 检查 core state 是否相容；
- 用 Wigner $3j/6j$ 系数处理角动量约化矩阵元；
- 用径向积分 $\langle R_f | r^k | R_i \rangle$ 给出 radial part；
- 对所有 channel pair 加权求和。

因此寿命结果同时依赖能级、角动量代数、径向波函数和 MQDT channel mixing。

total_decay 与显式终态求和的关系

两者使用相同的 partial rate 公式：

$$\Gamma_{\text{tot}} = \sum_f \Gamma_{i \rightarrow f}$$

差别只在“终态集合”：

- total_decay：内部自动调用 single_basis.fill，终态范围由当前实现固定。
- 显式求和：用户自己构造 $\{f\}$ ，再逐项调用 partial_decay。

什么时候应该显式构造终态

当自动终态搜索过大、过慢、扫到数值病态区，或者需要明确报告截断条件时，显式构造终态更可控。但这只解决“可控性”，不自动保证“完备性”；漏掉低能或其它 manifold 会直接导致寿命偏长。

对 ^{171}Yb 寿命计算的经验

以

$$|r\rangle = |65\ ^3S_1, F = 3/2, m_F = -3/2\rangle$$

为例：

- 态构造应使用：

```
get_state((65, 0, 1.5, -1.5), tt="vlfm").
```

- 原生 `total_decay` 会从很低的有效量子数区域开始自动生成终态。
- 在当前本地环境中，直接调用曾出现长时间不返回和低 ν 插值 / 非有限系数问题。
- notebook 中采用的显式 P 态终态范围：

$$\nu_{\min} = 3.0, \quad \nu_{\max} = \nu_i + 10.$$

- 这个范围会漏掉低 ν P 态，因此只能作为诊断性 subset，不能作为完整寿命。

早期结果: 65 3S_1 态的诊断性 subset

第一版 $\nu_{\min} = 3.0$ MQDT-only 终态求和结果:

温度	候选终态数	$\Gamma_{\text{tot}} / \text{s}^{-1}$	$\tau / \mu\text{s}$
0 K	1680	1.554823×10^3	643.160
300 K	1680	5.596472×10^3	178.684

- 这些寿命明显偏长, 说明这个终态集合少算了 channel。
- 后续改进的目标不是改变 partial_decay 公式, 而是补全终态集合。
- 这个表应理解为“缺 channel 前”的 baseline。

Fang 2001 benchmark: 低 n Yb 寿命

用 ^{174}Yb 的 $6sns^1S_0$ 和 $6snd^1D_2$ 序列做验证。Fang et al. 给出了室温 $\tau = 1/(A + B\rho)$ 计算值。

态	hybrid 显式 τ_{300K}	Fang τ_{300K}	$\Gamma_{\text{res}} / \text{s}^{-1}$	校准后 τ
$6s21s$	7.424	6.8	1.235×10^4	6.800
$6s22s$	8.417	8.3	1.671×10^3	8.300
$6s27s$	14.508	16.9	0	14.508
$6s21d$	15.218	7.8	6.249×10^4	7.800
$6s27d$	28.090	18.2	1.934×10^4	18.200

- s 系列在加入 NIST 低态后已接近 Fang 的量级和趋势;
- d 系列仍需要较大的 residual, 说明低 n perturbation / configuration mixing 仍未被显式模型完整覆盖。

最终目标态: explicit hybrid 结果

对

$$|r\rangle = |65\ ^3S_1, F = 3/2, m_F = -3/2\rangle$$

使用 NIST low P states + MQDT P states with $\nu \geq 12$:

项目	0 K	300 K
候选终态数	1542 = 84 NIST + 1458 MQDT	
$\Gamma_{\text{NIST}} / \text{s}^{-1}$	–	3.041537×10^3
$\Gamma_{\text{MQDT}} / \text{s}^{-1}$	–	4.261963×10^3
$\Gamma_{\text{explicit}} / \text{s}^{-1}$	3.262577×10^3	7.303500×10^3
$\tau_{\text{explicit}} / \mu\text{s}$	306.506	136.921

- 相比 MQDT-only subset, 300 K 显式速率从 5.60×10^3 升到 $7.30 \times 10^3 \text{ s}^{-1}$ 。
- 寿命仍长于实验值, 说明还存在未解析损失。

目标态：与实验寿命的 residual 校准

Muniz et al. 对相同 Rydberg 态在 typical tweezer depth 下给出：

$$\tau_{\text{exp}} \simeq 65(3) \mu\text{s}.$$

因此：

$$\Gamma_{\text{ref}} = \frac{1}{65 \mu\text{s}} = 1.538462 \times 10^4 \text{ s}^{-1}.$$

贡献	rate / s^{-1}
explicit hybrid	7.303500×10^3
residual sink	8.081115×10^3
calibrated total	1.538462×10^4

使用建议

做 branching 分析时看 explicit NIST/MQDT channels；做 gate loss / erasure 模型时再加入 residual sink，使总寿命匹配实验。

主要 300 K explicit 通道

排名	来源	终态	Γ / s^{-1}
1	NIST	$6^3P_2, F = 5/2, m_F = -5/2$	544.428
2	MQDT	$\nu = 65.08, L = 1, F = 5/2, m_F = -5/2$	510.435
3	MQDT	$\nu = 64.08, L = 1, F = 5/2, m_F = -5/2$	483.589
4	NIST	$6^3P_1, F = 3/2, m_F = -3/2$	277.815
5	MQDT	$\nu = 65.27, L = 1, F = 3/2, m_F = -3/2$	256.884
6	MQDT	$\nu = 65.34, L = 1, F = 1/2, m_F = -1/2$	218.292

- NIST low P 态负责大能量差的 spontaneous decay;
- 相邻 Rydberg P 态负责 300 K 下显著的 BBR-induced redistribution;
- 这两类通道必须同时保留, 才能接近真实 lifetime scale。

- ① 先确认态标记格式：Yb171 Rydberg MQDT 态优先用 `v1fm`。
- ② 记录环境参数：至少写明 T 、LDOS 是否为默认值。
- ③ 对 Yb171 MQDT 态，不直接依赖原生 `total_decay` 的一次性大范围搜索。
- ④ 优先使用 `windowed get_nearby` 构造高 Rydberg 终态，并显式报告 ν 范围。
- ⑤ 对低 lying final states，优先使用 NIST 能级并与 MQDT 候选去重。
- ⑥ 如果要匹配实验总寿命，把未解析部分作为 residual sink 单独列出。
- ⑦ 发表或 PR 中报告寿命时，同时报告：
 - 初态；
 - 终态截断；
 - 是否包含 BBR；
 - Γ 与 τ ；
 - 主要 partial decay channels。

总结

- rydcalc 提供从原子结构到相互作用、矩阵元和寿命的统一计算接口。
- total_decay 的本质是：

$$\text{自动生成终态} + \sum_f \text{partial_decay}(i, f).$$

- partial_decay 使用标准电偶极辐射速率公式，并加入温度相关的黑体光子占据数。
- 对复杂 MQDT 体系，寿命计算的可靠性不只取决于公式，还取决于终态集合是否数值稳定且物理完备。
- 我们不改 rydcalc 源码的前提下，引入了分窗 get_nearby、NIST 低态补充和 residual sink 校准。
- 对当前 ^{171}Yb 65S 目标态，explicit hybrid 给出 $\tau_{300K} = 136.921 \mu\text{s}$ ；加 residual sink 后可匹配 $65 \mu\text{s}$ 实验总寿命。